

Lenguajes de Programación - Examen Temas 1 y 2

Marzo de 2026. Soluciones.

Ejercicio 1 (3 puntos)

La empresa Acme vende diferentes tipos de artículos. Cada venta se modela como un ticket con una serie de líneas que son los productos comprados. Cada línea contiene el nombre del producto, el precio unitario y la cantidad de productos de ese tipo comprados. Las líneas tendrán un método `total()` que calculará el total de la línea multiplicando el precio unitario por la cantidad de producto.

Se desea modelar esta estructura con clases en Python, teniendo en cuenta que el ticket debe tener un método `total_lineas()` que iterará sobre cada una de las líneas pidiendo el total a cada una de ellas y devolviendo el total de cada línea una a una (función generadora).

Se valorará con 1.5 puntos las soluciones correctas que no utilicen para `total_lineas` función generadora. Se valorará con 3 puntos las soluciones correctas que usen función generadora.

Solución

```
class Linea:
    def __init__(self, nombre, precio, cantidad):
        self._nombre = nombre
        self._precio = precio
        self._cantidad = cantidad

    def total(self):
        return self._precio * self._cantidad

class Ticket:
    def __init__(self, lineas):
        self._lineas = lineas

    def total_lineas(self):
        for linea in self._lineas:
            yield linea.total()

l1 = Linea("leche", 0.98, 12)
l2 = Linea("Pan", 0.50, 2)

t = Ticket([l1, l2])
for total in t.total_lineas():
    print(total)
```

Ejercicio 2 (3 puntos)

Dada la siguiente función, modifíquela para utilizar la función `map` para obtener la lista `resultado` en lugar de iterar por la lista con el `for`.

```
def convert(lista_original):
    resultado = []
    for item in lista_original:
        resultado.append(len(item))
    return resultado
```

Solución

Se han considerado dos posibles soluciones válidas.

Con lambdas:

```
def convert(lista_original):
    return map(lambda i : len(i), lista_original)

print(list(convert([[1, 2], [1, 2, 3]])))
```

Sin lambdas:

```
def convert(lista_original):
    return map(len, lista_original)

print(list(convert([[1, 2], [1, 2, 3]])))
```

Ejercicio 3 (4 puntos. Respuesta correcta: 0,4 puntos, respuesta incorrecta: -0,12 puntos, sin respuesta: 0 puntos)

Marque la respuesta elegida (a, b, c o d) en la siguiente tabla. Si se equivoca, táchela y marque la escogida debajo. No se considerarán respuestas que no estén en la tabla.

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

Indique la respuesta correcta:

- Las funciones lambda no pueden recibir parámetros
 - Verdadero
 - Falso, pueden recibir un único parámetro
 - Falso, pueden recibir tantos parámetros como se requiera**
 - Falso deben recibir siempre al menos un parámetro
- Una función `f` definida dentro de un fichero `mf.py` que se usa desde otro fichero python en la misma carpeta como `f(...)`, tendría que importarse con:
 - `import mf.py`
 - `import mf`
 - `from mf import mf`
 - `from mf import f`**
- ¿Cómo podemos importar un módulo `mimodulo` indicando un alias para referirnos a él?
 - `import mimodulo as mm`**
 - `from mimodulo import mm`
 - `from mimodulo as mm`
 - Todas son correctas
- Dado el siguiente fragmento de código, ¿de qué tipo es `a`?


```
for x, y in a.items():
    print(f"({x},{y})")
```

 - Lista (list)

- b) Tupla (tuple)
- c) **Diccionario (dict)**
- d) Conjunto (set)

5. ¿Cuántos argumentos posicionales tiene la siguiente función?

```
def func(a, b, c, d, *e, f, g):  
    ...
```

- a) Dos
- b) Cuatro
- c) **Cinco** (*e es el último parámetro posicional)
- d) Uno

6. Dado el siguiente fragmento de código indica qué imprimiría por pantalla:

```
def inverso(n):  
    return 1/n  
def wrapper(func, arg):  
    return func(arg)  
print(wrapper(inverso, 5))
```

- a) None
- b) 5
- c) **0.2**
- d) Error: attribute n not defined

7. ¿Cómo se invoca al constructor de la clase padre desde el de la clase hija?

- a) `super(...)`
- b) `super.__init__(...)`
- c) `super()`
- d) `super().__init__(...)`

8. Dada la definición `a = (1, "Hola", [1, 2, 3])`, ¿qué imprime por pantalla `print(a[2])`?

- a) Hola
- b) **[1, 2, 3]**
- c) 1
- d) 3

9. Indica la definición correcta. La instrucción `yield 10`:

- a) Devuelve 10 al que llamó a la función que contiene el `yield` y termina la ejecución de dicha función
- b) Devuelve 10 al que llamó a la función que contiene el `yield` y termina la ejecución del programa
- c) **Devuelve 10 al que llamó a la función que contiene el `yield` y pausa la ejecución de la función hasta la siguiente llamada, continuando a partir de la sentencia que sigue a `yield`**
- d) Ninguna de las anteriores es correcta

10. ¿El siguiente `for` cuantas veces ejecuta su cuerpo?

```
for x in [x**2 for x in (1, 2, 3)]:  
    print(x)
```

- a) 0
- b) 1
- c) 2
- d) **3** (hay 3 elementos en la tupla)